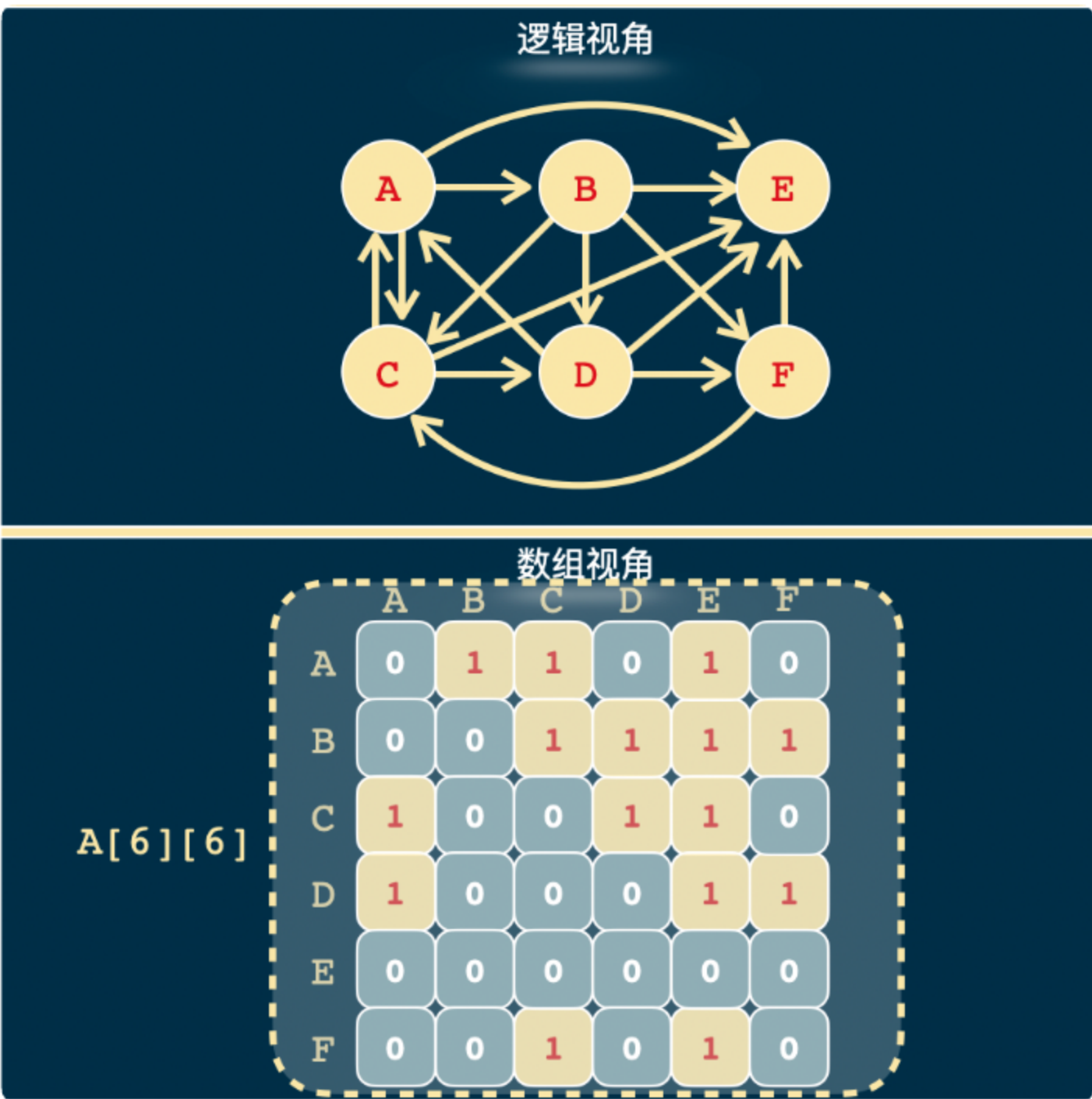


6-2 图的存储结构

邻接矩阵法

画法



代码表示

```
#define MaxInt 32767 //表示极大值, 即∞
#define MVNum 100 //最大顶点数
typedef struct{
    VerTexType vexs[MVNum]; //顶点表
    ArcType arcs[MVNum][MVNum]; //邻接矩阵
    int vexnum, arcnum; //图的当前点数和边数
}AMGraph;
```

优点

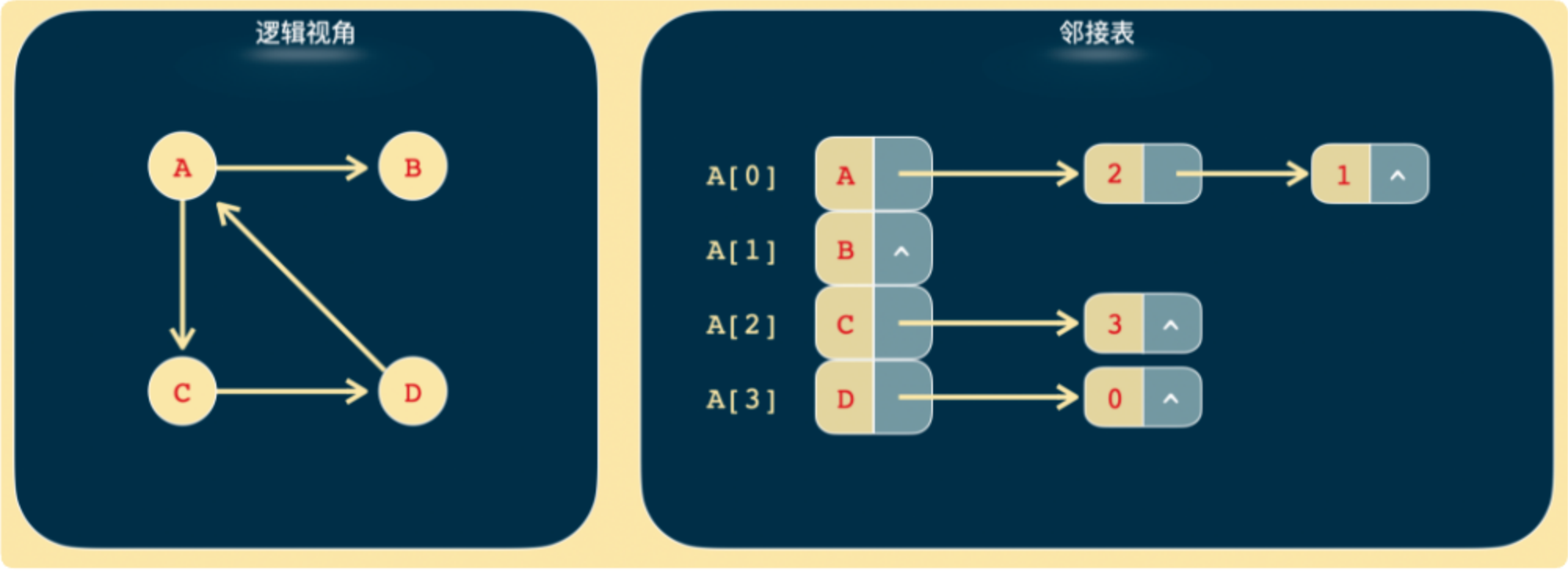
- 便于判断两个顶点间是否有边
- 便于计算各个顶点度

缺点

- 不便增加和删除节点
- 不便统计边的数目
- 空间复杂度高

邻接表法

画法



优点

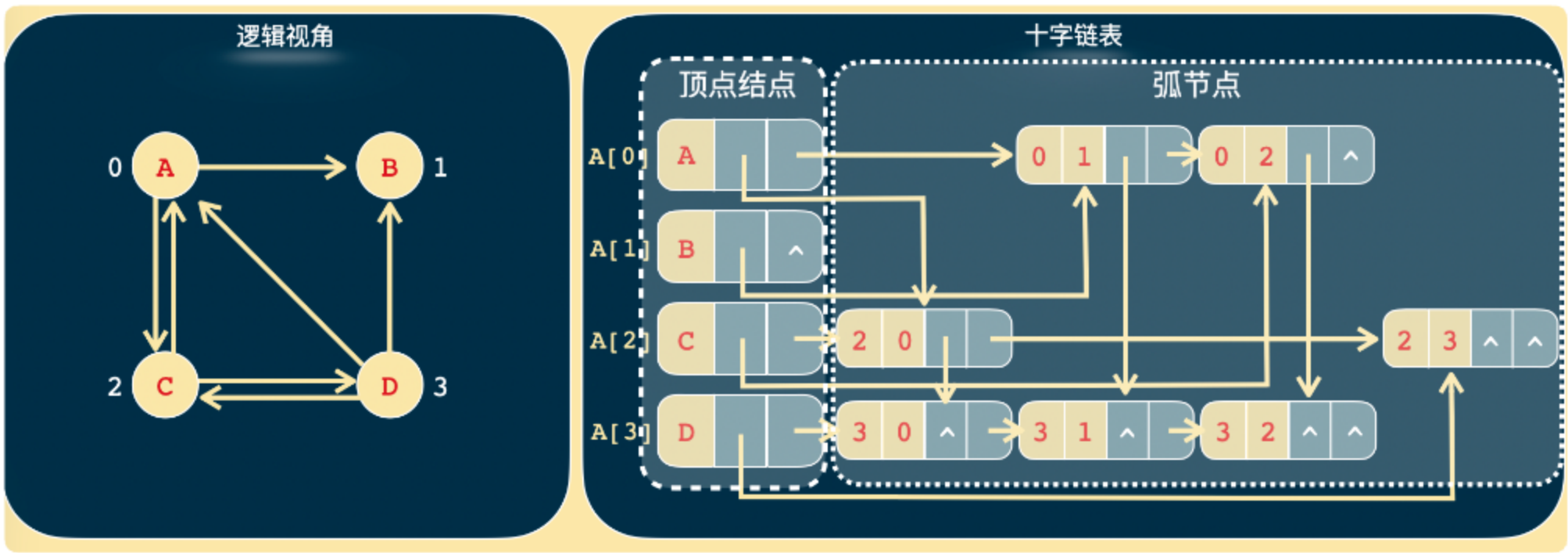
- 便于增加删除节点
- 便于统计边的数目
- 空间效率高

缺点

- 不便判断顶点之间是否有边
- 不便计算有向图各个顶点的度

十字链表 (只适用于有向图)

画法

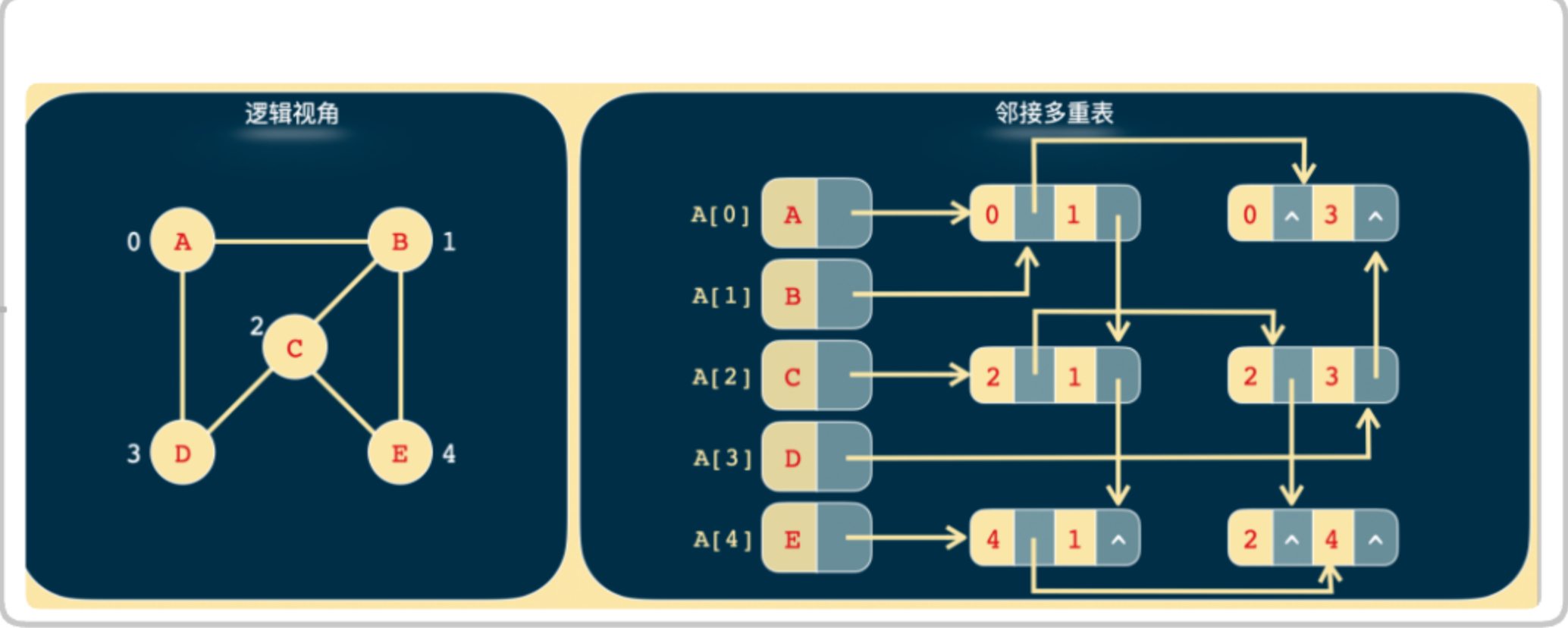


优点

十字链表结合了邻接表和逆邻接表, 可以方便地计算顶点Vi的入度和出度/寻找依附于某一结点的所有弧

邻接多重表 (只适用无向图)

画法



边节点由5部分组成

- ivex:边依附第1个顶点的编号
- ilink:依附于顶点i的下一条边
- jvex:边依附第2个顶点的编号
- jlink:依附于顶点j的下一条边
- info:边的信息(如权重, 图中未画出)

邻接多重表中, 每条边只会被存储一次, 且所有依附于同一结点的边都被串联在一个链表中, 便于计算结点的度/寻找依附于某一结点的所有边

四种方法的对比

	邻接矩阵法	邻接表法	十字链表	邻接多重表
空间复杂度	$O(V^2)$	无向图 $O(V^2 + 2E)$ 有向图 $O(V^2 + E)$	$O(V + E)$	$O(V + E)$
找相邻边	遍历对应行/列 复杂度 $O(V)$	有向图找入度 要遍历整个邻接表	方便	方便
删除边或点	删边方便 删点麻烦	无向图删边点都不方便	方便	方便
适用范围	稠密图	稀疏图	只能有向图	只能无向图
是否唯一	唯一	不唯一	不唯一	不唯一